

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce
Department of Information Technology (Bsc.IT Semester III)
SCALA for DS
Practical

<i>Roll No. : SY50</i>	<i>Name: Nitesh Yadav</i>
<i>Class:SYIT</i>	<i>Date of Submission :25/08/2026</i>

21).Join two CSV files in Spark DataFrames based on a common column and write the output to a file.

Input:

```
object Pract21 extends App {  
  val s = SparkSession.builder()  
    .appName("JoinTwoCSV")  
    .master("local[*]")  
    .getOrCreate()  
  
  val d1 = s.read  
    .option("header", "true")  
    .csv("File1.csv")  
  
  val d2 = s.read  
    .option("header", "true")  
    .csv("File2.csv")  
  
  val j = d1.join(d2, Seq("id"), "inner")  
  
  j.show()  
  
  // Save joined output to CSV  
  j.write.option("header", "true").csv("output_result")  
  
  s.stop()  
}
```

Output:

```
+---+-----+-----+
| id|  name|score|
+---+-----+-----+
|  1|  Rani|   85|
|  2|Sachin|   40|
|  3| Priya|   78|
+---+-----+-----+
```

22) Set up Apache Spark locally and count the frequency of words in a text file.

Input:

```
import org.apache.spark.sql.SparkSession

object Parct22 extends App
{
  val s = SparkSession.builder()
    .appName("WordCountExample")
    .master("local[*]")
    .getOrCreate()

  val r = s.sparkContext.textFile("input.txt")
  val w = r.flatMap(line => line.split("\\s+"))
  val wo = w.map(word => (word, 1))
  val wc = wo.reduceByKey(_ + _)

  wc.collect().foreach(println)
  s.stop()
}
```

Output:

```
(is,2)
(easy,1)
(powerful,1)
(Spark,2)
```

23).Filter rows in a CSV file using Spark DataFrames where a numeric column exceeds a threshold.

Input:

```

import org.apache.spark.sql.SparkSession

object Demo extends App {

  val spark = SparkSession.builder()
    .appName("FilterCSVExample")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val df = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("src/main/resources/superstore.csv")

  val f = df.filter($"Sales" > 5000)

  println("Filtered Data:")
  f.show()

  spark.stop()
}

```

Output:

```

+-----+-----+-----+
|OrderID| Product|Sales|
+-----+-----+-----+
|      2|  Laptop| 7500|
|      4|Keyboard| 6200|
+-----+-----+-----+

```

24).Perform a group-by operation in Spark DataFrames to compute the average of each group.

Input:

```

import org.apache.spark.sql.SparkSession
import org.apache.spark.sql.functions._

object Demo extends App { new {

  val spark = SparkSession.builder()
    .appName("AverageSales")
    .master("local[*]")
    .getOrCreate()

  val df = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("src/main/resources/superstore.csv")

  val result = df.groupBy("Category")
    .agg(avg("Sales").alias("Average Sales"))

  println("Average Sales by Category:")
  result.show()

  spark.stop()
}

```

Output:

```

+-----+-----+
|      Category|Average Sales|
+-----+-----+
|Office Supplies|      4000.0|
|      Furniture|      6000.0|
|    Technology|      7000.0|
+-----+-----+

```